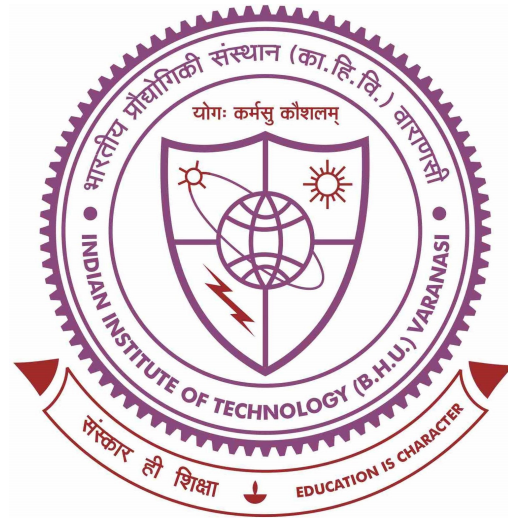


Reward Decomposition in MARL—LOMA_Q Algorithm



Report submitted in partial fulfilment for

EXPLORATORY PROJECT OF SECOND YEAR B.TECH

in

COMPUTER SCIENCE AND ENGINEERING

by

G UNUREDDY, K V K VARUN, V N V S S S JAYADITHYA

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

INDIAN INSTITUTE OF TECHNOLOGY

(BANARAS HINDU UNIVERSITY)

VARANASI – 221 005

ROLL NUMBERS
21075035, 21075044,
21075092

YEAR OF SUBMISSION
2023

CERTIFICATE

It is certified that the work contained in the report titled **Reward Decomposition in MARL—LOMAQ Algorithm** by **G Unureddy, K V K Varun, V N V S S Jayadithya** has been carried out under my/our supervision and that this work has not been submitted elsewhere for a degree.

It is further certified that the student has fulfilled all the requirements of Comprehensive Examination, Candidacy for the **Exploratory project of Second Year B.Tech.**

Supervisor

Dr. Lakshmanan Kailasam

Department of Computer Science and Engineering

Indian Institute Of Technology

(Banaras Hindu University)

Varanasi – 221 005

DECLARATION BY THE CANDIDATE

We, **G Unureddy, K V K Varun, V N V S S Jayadithya**, certify that the work embodied in this report is our own bona fide work and carried out by us under the supervision of **Dr. Lakshmanan Kailasam** from **January 2023** to **May 2023**, at the **Department of Computer Science and Engineering**, Indian Institute of Technology (Banaras Hindu University), Varanasi. The matter embodied in this report has not been submitted for the award of any other degree/diploma. We declare that we have faithfully acknowledged and given credits to the research workers wherever their works have been cited in our work in this thesis. We further declare that we have not willfully copied any other's work, paragraphs, text, data, results, *etc.*, reported in journals, books, magazines, reports dissertations, theses, *etc.*, or available at websites and have not included them in this report and have not cited as our own work.

Date: May 10, 2023
Place: Varanasi, India

Signature of the Student

G. Unureddy
K. Varun
V. Jayadithya

CERTIFICATE BY THE SUPERVISOR

It is certified that the above statement made by the student is correct to the best of my/our knowledge.

Supervisor

Dr. Lakshmanan Kailasam

Department of Computer Science and Engineering

Indian Institute Of Technology

(Banaras Hindu University)

Varanasi – 221 005

Signature of Head of Department

COPYRIGHT TRANSFER CERTIFICATE

Title of the Report: Reward Decomposition in MARL—LOMAQ Algorithm

Name of the Student: G Unureddy, K V K Varun, V N V S S Jayadithya

COPYRIGHT TRANSFER

The undersigned hereby assigns to the Indian Institute of Technology (Banaras Hindu University), Varanasi all rights under copyright that may exist in and for the above report submitted for the Exploratory project of Second Year B.Tech in Computer Science and Engineering.

Date: May 10, 2023
Place: Varanasi, India

Signature of the Student

G. Unureddy
K. Varun
V. Jayadithya

Note: However, the author may reproduce or authorize others to reproduce material extracted verbatim from the report or derivative of the report for author's personal use provided that the source and the Institute's copyright notice are indicated.

ACKNOWLEDGEMENT

We would like to thank Dr. Lakshmanan Kailasam for constantly supporting us throughout the duration of the project.

Preface

Reinforcement Learning is a branch of machine learning that deals with training an agent to make decisions based on trial and error interactions with an environment. The agent learns to take actions that maximize a reward signal received from the environment, while also taking into account the long-term consequences of its actions. Unlike supervised learning, where the model is trained on labeled data, and unsupervised learning, where the model is trained on unlabeled data, RL involves learning from a feedback signal that is delayed, sparse, and not always optimal. RL algorithms are well-suited for applications where the optimal action may depend on the current state of the environment and can change over time.

Reinforcement Learning has many potential applications in various fields and industries, but there are still challenges to overcome, such as dealing with high-dimensional state and action spaces, sample efficiency etc,. Multi-Agent Reinforcement Learning (MARL) is a subfield of machine learning concerned with training a group of agents to interact with an environment together in order to maximize a cumulative reward. In MARL, agents learn to coordinate their actions by communicating and sharing information with each other, allowing them to solve tasks that are too complex for a single agent to handle.

In this project we have used novel value-based multi-agent algorithm, LOMAQ, for cooperative multi-agent reinforcement learning (MARL) that incorporates local rewards in the Centralized Training Decentralized Execution (CTDE) paradigm, presented in the paper [Locality Matters: A Scalable Value Decomposition Approach for Cooperative Multi-Agent Reinforcement Learning](#) in various test environments.

Contents

1	Abstract	1
2	Introduction	2
3	Reward Decomposition	4
4	LOMAQ Algorithm	8
4.1	Algorithm	8
5	Implementation and Results	10
5.1	Implementation	10
5.2	Results	11
6	Conclusion and Future Work	13

List of Figures

2.1	Multi Agent Reinforcement Learning	
	Source: https://www.mdpi.com/1996-1073/13/1/123	2
3.1	A pictorial representation of CoMARL	6
4.1	Architecture of LOMA <i>Q</i> algorithm	
	The agent's states s_i are fed into the utility networks U_i^θ , which are then forwarded as inputs into F_J^θ	9
5.1	Graphical Statistics of test 2 throughout the run	11
5.2	Test returns for the Coupled-Multi-Cart-Pole and the Bounded-Cooperative- Navigation environments.	12

List of Tables

5.1	Test Results of some sample test cases	11
-----	--	----

Chapter 1

Abstract

Multi-Agent Reinforcement Learning (MARL) is a field in reinforcement learning that focuses on studying multiple agents' behavior in a single environment. Applying single-agent reinforcement learning techniques to MARL can be done in various ways. Applying single-agent reinforcement learning techniques to MARL can be done in various ways. Multi-agent reinforcement learning is modeled as a particular kind of Markov decision process (MDP), just like single-agent reinforcement learning. Each agent acts to further its interests and is driven by its rewards. In some contexts, these interests conflict with those of other agents, which leads to complicated group dynamics. As environments grow in size, effective credit assignment becomes increasingly more complicated and often results in infeasible learning times.

The LOMAQ algorithm proposed by Zohar, Mannor, Tennenholtz (2021) — Locality Matters: A Scalable Value Decomposition Approach for Cooperative Multi-Agent Reinforcement Learning [Hebrew University and Nvidia Research], provides a way to incorporate local rewards in the Centralised Training Decentralised Execution (CTDE) paradigm.

Chapter 2

Introduction

Reinforcement learning is concerned with the agent taking suitable actions on the trade-off between exploration and exploitation of the tasks it discovers and the tasks it has already discovered, which maximizes its value or the cumulative reward. Multi-Agent Reinforcement Learning, also known as MARL, is an extension of the general reinforcement learning problem. MARL deals with reward signaling in the case of multiple agents doing concurrent actions in the same environment simultaneously. This problem poses a difficulty in assigning the reward signal to each agent when the change affects the whole environment. Research on MARL has gained much attention in recent years, with examples in the Star-Craft multi-agent challenge (Vinyals et al. 2019) and traffic control (Chuet al. 2019).

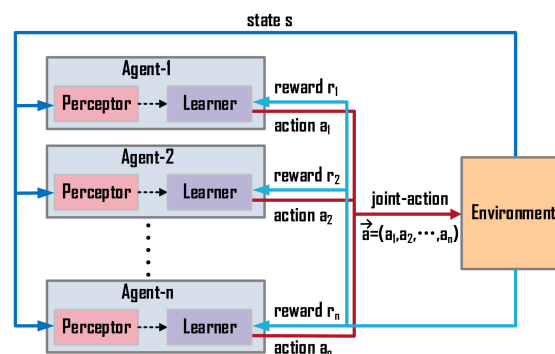


Figure 2.1: Multi Agent Reinforcement Learning
Source: <https://www.mdpi.com/1996-1073/13/1/123>

The CTDE (Centralized Training with Decentralized Execution) paradigm is commonly used in Cooperative Multi-Agent Reinforcement Learning (MARL). In this paradigm, a centralized critic is used to train decentralized actors.

In the CTDE paradigm, a centralized critic estimates the value of a joint action taken by multiple agents in the environment. The critic takes as input the states and actions of all agents and produces a single value estimate. The critic is trained using standard RL algorithms, such as Q-learning or actor-critic. On the other hand, the decentralized actors operate independently in the environment and make decisions based on their local observations. Each actor has a policy that maps observations to actions. These policies are typically trained using gradient descent methods such as policy gradients. During training, the centralized critic provides feedback to the decentralized actors by evaluating their joint actions. The actors use this feedback to update their policies to improve their performance. In this way, the CTDE paradigm combines the benefits of both centralized and decentralized approaches, allowing agents to learn effective joint behaviors while maintaining the scalability and robustness of decentralized execution.

The LOMAQ algorithm uses local rewards as the principal component for the reward decomposition. Local rewards are critical for effective learning in scalable settings. For example, let us consider the problem of the mentor of a team of players playing a first-person shooter game in a competition. If a particular player is consistently getting killed by the other team, punishing that player directly (and possibly neighboring players) with targeted feedback may be far more effective than punishing the entire team with general feedback. The latter will often confuse players, believing they are not doing the correct action.

Even though local rewards are helpful, naive training with them may produce greedy agents that may not work well with other agents. Approaches that attempt to take advantage of local reward systems in the environment frequently need higher levels of cooperation and typically turn to training with global rewards. But relying solely on the decomposition global rewards is a slow process and is thus ineffective.

Chapter 3

Reward Decomposition

Let us define a MAMDP (Multi Agent Markov Decision Process) as follows:

$\mathcal{M} = (\mathcal{G}, \mathcal{S}, \mathcal{A}, P, r, \gamma)$, where

- $\mathcal{G} = (\mathcal{V}, \mathcal{E})$ is the undirected graph of agents, where $\mathcal{V} = \{1, 2, \dots, n\}$ and $\mathcal{E} \subseteq \mathcal{V} \times \mathcal{V}$.

Describes that there are n agents in the system, each represented by a node in the graph, and the edges between nodes represent the relationships or interactions between agents.

- $\mathcal{S} = \times_{i=1}^n \mathcal{S}_i$

Represents the global state space, where $\mathcal{S}_i$ is the state space of agent i . The global space $\mathcal{S}$ represents the joint state space of all agents in the system.

- $\mathcal{A} = \times_{i=1}^n \mathcal{A}_i$

Represents the global action space, where $\mathcal{A}_i$ is the action space of agent i . The global action space represents the joint action of all agents in the system.

- $P : \mathcal{S} \times \mathcal{S} \times \mathcal{A} \mapsto [0, 1]$

Represents the global transition function, which describes the probability of transitioning from the current global state to an other global state. It maps from the

current global state, next global state which would be generated by the joint action $\mathcal{A}$ and the joint action to a value between $[0, 1]$.

- $r : \mathcal{S} \times \mathcal{A} \mapsto \mathbf{R}$

Represents the global reward function, which provides a reward for each joint action taken in a global state. This function takes as input the current global state and the joint action taken by all agents.

- $\gamma \in (0, 1)$ is the discount factor, which determines the importance of future rewards relative to immediate rewards.

Agent $i \in \mathcal{V}$ is associated with the underlying graph $\mathcal{G}$, state s_i , and action a_i . For a set $B \subseteq V$, we define s_B and a_B as the subset of agent states and actions in B , respectively. At time t , the environment is at state s and agents take an action a . The environment returns a reward r and transitions to state s_0 according to the factored dynamics $P(s_0|s, a)$, which is a conditional probability distribution over the next state s_0 given the current state s and action a . The factored dynamics function is defined as a product of local transition probabilities $P(s_0|s, a) = \prod_{i \in \mathcal{V}} P_i(s_{0i}|s_{N(i)}, a_i)$, where $N(i)$ is the neighborhood of agent i in the graph $\mathcal{G}$.

We define a global Markov policy π as a mapping that assigns a probability to each joint action a given a global state s . In other words, $\pi(a|s)$ is the probability to choose action $a = (a_0, a_1, a_2, \dots, a_n)$ at state $s = (s_0, s_1, s_2, \dots, s_n)$.

The value of policy π starting at a state s and taking action a is defined as the expected sum of discounted rewards, where the expectation is taken over the transitions of the system according to the dynamics $\mathcal{P}$ and the policy π . This is denoted as $Q^\pi(s, a)$ and is a function of the state-action pair.

The value function $v^\pi(s)$ is defined as the expected value of $Q^\pi(s, a)$ under the policy π , where the expectation is taken over the joint action a sampled from the policy π .

The optimal value and optimal policy are denoted as $v^*(s)$ and π^* , respectively. The optimal value function $v^*(s)$ is the maximum expected sum of discounted rewards over

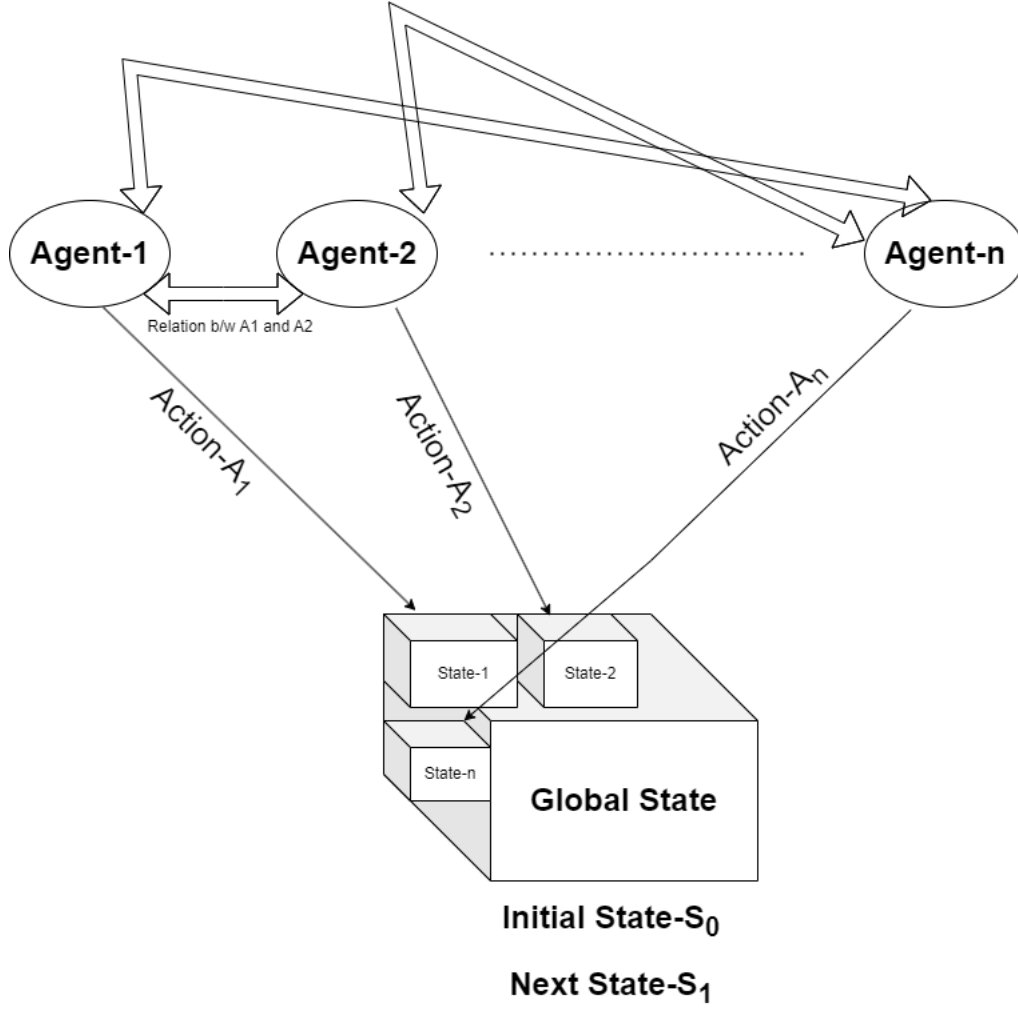


Figure 3.1: A pictorial representation of CoMARL

all policies, and the optimal policy π^* is the policy that achieves this maximum value.

If we assume that the reward is additively decomposable, i.e.,

$$\exists \{r_i : \mathcal{S}_i \times \mathcal{A}_i \mapsto \mathbf{R}\}_{i \in \mathcal{V}} | r(s, a) = \sum_{i=1}^n r_i(s_i, a_i) \forall s \in \{s_1, s_2, \dots, s_n\} \wedge a \in \{a_1, a_2, \dots, a_n\}$$

, then we can define a partial Q function of π as

$$Q_i^\pi(s, a) = \mathbf{E}_\pi \left[\sum_{t=0}^{\infty} \gamma^t r_i(s(t), a(t)) | s(0) = s, a(0) = a \right]$$

We can observe that this type of decomposition is always possible, at least by setting $r_1 = r$ and $r_i = 0$ if $i \geq 2$. In the LOMA Q algorithm, the rewarded decomposition for every agent is supposed to be dependent only on the local state of the agent and its action,

as assumed.

Chapter 4

LOMAQ Algorithm

The Local Multi-Agent Q -learning algorithm, (LOMAQ) is based on the maximisation of the Q -summation defined in Chapter-3 and an assumption described below, which is also known as the Monotonic Decomposition:

The method assumes the existence of a partition $\mathcal{P}$ of the set of agents $\mathcal{V}$, along with the utility functions $U_i^\pi : S_i \times \mathcal{A}_i \mapsto \mathbf{R}$, $i \in \mathcal{V}$ and partition functions $\{F_J^\pi : \mathbf{R}^n \mapsto \mathbf{R}\}$, $J \in P$. Each utility function U_i^π associates a value with the state-action pair (s_i, a_i) of agent i , and each partition function F_J^π maps a global state-action vector to a scalar value for each partition J . Specifically, for all $J \in P$, the partition function F_J^π is defined as the sum of the local utility functions for agents in J , i.e.,

$$F_J^\pi(\mathbf{U}(s, a)) = \sum_{i \in J} Q_i^\pi(s, a)$$

and, $\nabla_{\mathbf{U}} F_J^\pi \geq 0$, where $\mathbf{U}(s, a) = (U_1^\pi(s_1, a_1), U_2^\pi(s_2, a_2), \dots, U_n^\pi(s_n, a_n))^T$

4.1 Algorithm

Algorithm 1 LOMAQ algorithm

```

1: Input: Partition  $\mathcal{P}$  of  $\mathcal{V}$ , exploration parameter  $\epsilon$ 
2: Initialise:  $F_J(U(s', a')) = 0 \quad \forall J \in \mathcal{P}$ 
3: for  $t$  in  $\text{range}(1, n)$ :
    Take action  $a$ 
    Observe  $s'$  and local rewards  $\{r_J\}_{J \in \mathcal{P}}$ 
     $a'_{\text{greedy}} \in (\arg \max_{a'_i} U_i(s'_i, a'_i))_{i \in \mathcal{V}}$ 
     $a' \leftarrow \begin{cases} \text{random action,} & \text{w.p. } \epsilon \\ a'_{\text{greedy}}, & \text{w.p. } 1 - \epsilon \end{cases}$ 
    for  $J \in \mathcal{P}$  do:
         $F_J(U(s, a)) \xleftarrow{\alpha t} r_J(s, a) + \gamma F_J(U(s', a'))$ 
        Project  $F_J$  to the set  $\{f : \mathbf{R}^n \rightarrow \mathbf{R} \text{ s.t. } \nabla f \geq 0\}$ 
    end for
end for
    
```

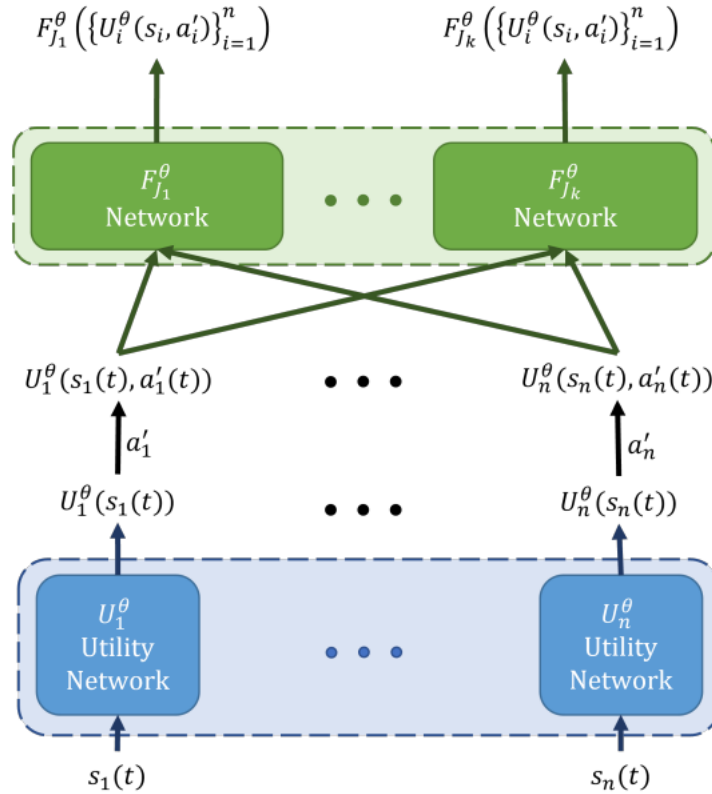


Figure 4.1: Architecture of LOMAQ algorithm

The agent's states s_i are fed into the utility networks U_i^θ , which are then forwarded as inputs into F_J^θ

Chapter 5

Implementation and Results

5.1 Implementation

We have tested the algorithm in different environments like payoff-matrix, traffic signal control, coupled multi cart pole etc., using LOMAQ and other algorithms as well algorithms like iql, qmix etc. The code implementation relies on a set of hyper-parameters. There are 4 different types of configuration files, and they are described as follows:

- Default Configuration

This sets the default parameters for all runs. It serves as a baseline and is overridden by any other configuration file. An example of a parameter in this file could be batch size, which generally remains similar for all algorithms.

- Environment Configuration

This contain parameters specifically relevant to running the environment. For instance, the num agents parameter can be defined here.

- Algorithm Configuration

Holds the parameters specific to the current algorithm being used. An example could be the learning rate parameter, which determines the rate at which the algorithm learns.

- Test Configuration

These provide information for a particular test scenario. If one wishes to run a specific algorithm against different num agents, a separate test file should be used. Each test file represents a series of runs. It's important to note that the Test Configuration file overrides all other configurations, allowing for precise customization in testing scenarios.

5.2 Results

We have implemented algorithms like coma, graphmix, iql, qmix etc., along with LOMAQ to compare the performance of the algorithms in different testing environments. Some of the test results in the Multi particle environment are described in the table below:

Test Number	Algorithm	Environment	Test Return Mean	Test Std Dev
1	qtran	Multi particle	59.23	23.21
1	iql	Multi particle	58.43	24.25
1	iql-local	Multi particle	93.15	34.49
1	qmix	Multi particle	65.52	25.32
1	lomaq	Multi particle	66.11	21.50
2	lomaq	Multi particle	63.98	19.20

Table 5.1: Test Results of some sample test cases

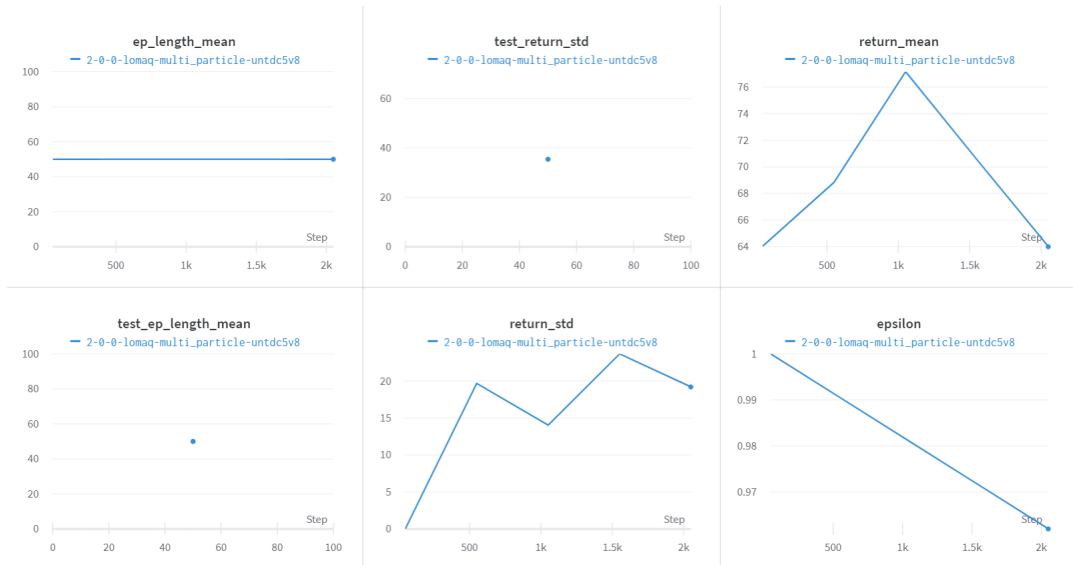


Figure 5.1: Graphical Statistics of test 2 throughout the run

After testing on almost 50 test cases in different environments and algorithms, we have found out that the LOMA Q algorithm gives better results in most cases in environments like multi cart pole, bounded cooperative navigation etc,. The following graph shows the comparison between different algorithms' performance in the fore mentioned environments:

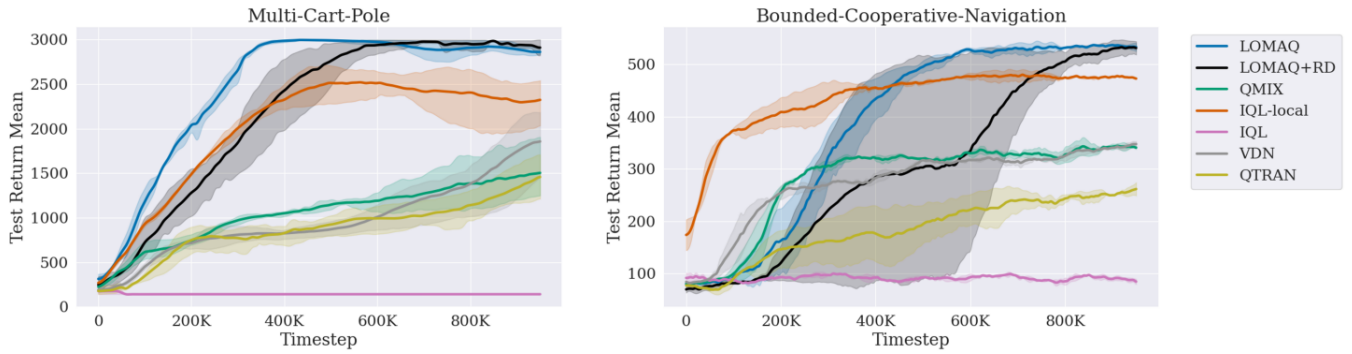


Figure 5.2: Test returns for the Coupled-Multi-Cart-Pole and the Bounded-Cooperative-Navigation environments.

The shaded area in the graphs represent the standard deviation of the return. So, we can conclude that LOMA Q performs better in these cases and also has a low standard deviation than algorithms like iql, qtran etc,.

Chapter 6

Conclusion and Future Work

In this study, we used local, partition-based value functions to solve the cooperative MARL credit assignment problem. We built a value-based strategy using the QSM condition and a monotonic decomposition of utilities, which successfully simplified the problem. We demonstrated that local rewards are extremely advantageous both when given and when implicitly learned from a global reward. These significantly increased efficiency and speed of convergence, indicating that local structures can be effectively exploited to enhance MARL algorithms.

In this study, we made the assumption that a static dependency graph $\mathcal{G}$ would be supplied during training. These presumptions are often restrictive. By observing such dynamic dependencies between agents through interaction with the environment, we hope to further generalise the approach. We look to generalize the LOMA Q algorithm to automatically identify effective decompositions.

References

1. Abbasi-Yadkori, Y.; Pal, D.; and Szepesvári, C. 2011. Improved algorithms for linear stochastic bandits. *Advances in neural information processing systems* 24: 2312–2320.
2. Son, K.; Kim, D.; Kang, W. J.; Hostallero, D. E.; and Yi, Y. 2019. Qtran: Learning to factorize with transformation for cooperative multi-agent reinforcement learning. In *International Conference on Machine Learning*, 5887–5896. PMLR.
3. Wang, J.; Ren, Z.; Liu, T.; Yu, Y.; and Zhang, C. 2020. QPLEX: Duplex Dueling Multi-Agent Q-Learning. In *International Conference on Learning Representations*.
4. Qu, G.; Lin, Y.; Wierman, A.; and Li, N. 2020. Scalable Multi-Agent Reinforcement Learning for Networked Systems with Average Reward. In *Thirty-fourth Conference on Neural Information Processing Systems*.
5. Zohar, R; Mannor, S; Tennenholtz G; 2020. Locality Matters: A Scalable Value Decomposition Approach for Cooperative Multi-Agent Reinforcement Learning
6. Lin, Y.; Qu, G.; Huang, L.; and Wierman, A. 2020a. Multiagent reinforcement learning in time-varying networked systems. *arXiv preprint arXiv:2006.06555* .
7. Kraemer, L.; and Banerjee, B. 2016. Multi-agent reinforcement learning as a rehearsal for decentralized planning. *Neurocomputing* 190: 82–94.
8. Russell, S. J.; and Zimdars, A. 2003. Q-decomposition for reinforcement learning agents. In *Proceedings of the 20th International Conference on Machine Learning (ICML-03)*, 656–663.

9. Rashid, T.; Samvelyan, M.; Schroeder, C.; Farquhar, G.; Foerster, J.; and Whiteson, S. 2018. Qmix: Monotonic value function factorisation for deep multi-agent reinforcement learning. In *International Conference on Machine Learning*, 4295–4304. PMLR.
10. Juozapaitis, Z.; Koul, A.; Fern, A.; Erwig, M.; and DoshiVelez, F. 2019. Explainable reinforcement learning via reward decomposition. In *IJCAI/ECAI Workshop on Explainable Artificial Intelligence*.
11. Sunehag, P.; Lever, G.; Gruslys, A.; Czarnecki, W. M.; Zambaldi, V. F.; Jaderberg, M.; Lanctot, M.; Sonnerat, N.; Leibo, J. Z.; Tuyls, K.; et al. 2018. Value-Decomposition Networks For Cooperative Multi-Agent Learning Based On Team Reward. In *AAMAS*.
12. van Seijen, H.; Fatemi, M.; Romoff, J.; Laroche, R.; Barnes, T.; and Tsang, J. 2017. Hybrid reward architecture for reinforcement learning. In *Proceedings of the 31st International Conference on Neural Information Processing Systems*, 5398–5408.
13. Chu, T.; Wang, J.; Codeca, L.; and Li, Z. 2019. Multi- ‘ agent deep reinforcement learning for large-scale traffic signal control. *IEEE Transactions on Intelligent Transportation Systems* 21(3): 1086–1095.